

NetNomos 代码逻辑与核心算法 图解阅读指南

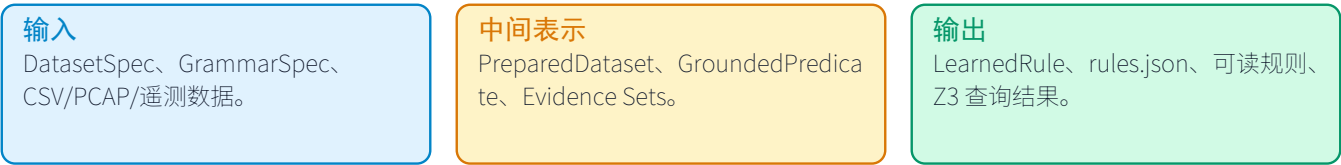
用博客文章的方式解释项目：从数据流、代码结构、谓词生成，到 minimal hitting set 的公式推导和 Z3 验证。适合有神经网络基础、但第一次接触符号规则挖掘的研究生快速入门。

先建立正确期待

当前仓库主要实现论文中的 Rule Learning，以及规则解释、验证和蕴含查询；LLM 过滤和 GPT-2 token 级 SMT 强制生成不是这个代码包的主流程。

1. 一句话理解：网络数据中的隐藏规则挖掘

NetNomos 可以先被理解为一个逻辑规则挖掘器。它读取网络数据和两份配置：DatasetSpec 描述数据如何被解释，GrammarSpec 描述允许搜索哪些谓词。系统生成候选谓词，再把这些谓词组合成和数据一致的逻辑规则。



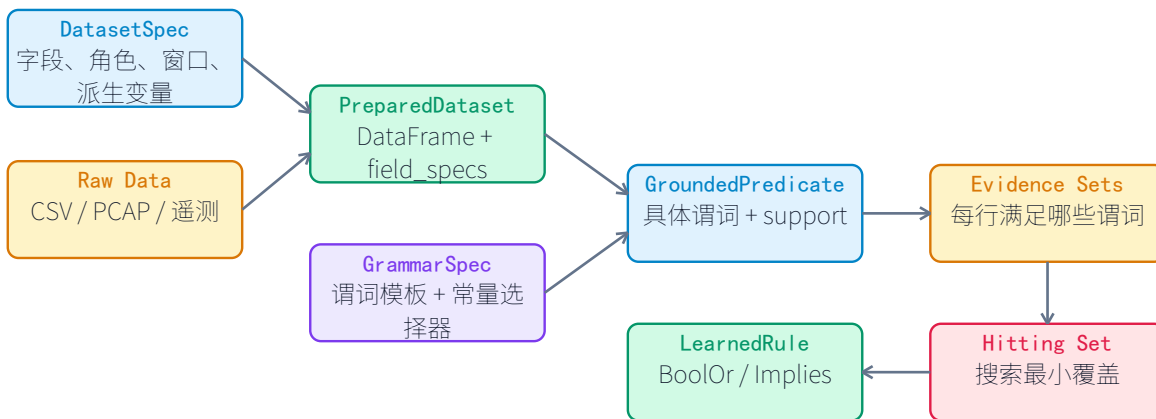
论文三阶段与当前仓库



因此，读这个仓库时不要寻找深度学习训练循环；真正的主角是配置化逻辑语法、谓词投影、DataFrame 求值、hitting-set 搜索和 Z3 推理。

2. 数据流：从配置和原始数据到规则产物

这张图是阅读整个项目最重要的地图。只要能沿着箭头说清楚每个对象是什么，后续代码就不会迷路。



代码调用链

```
NetNomosMiner.fit()  
-> prepare_dataset()  
-> generate_predicates()  
-> HittingSetLearner.fit() / EntropyTreeLearner.fit()
```

3. 代码结构：先读总控层，再读算法层



阅读顺序建议

第一遍只跟 NetNomosMiner.fit() 的调用链；第二遍看 DatasetSpec 和 GrammarSpec 怎样约束搜索空间；第三遍再进入 hitting-set 搜索和 Z3 推理细节。

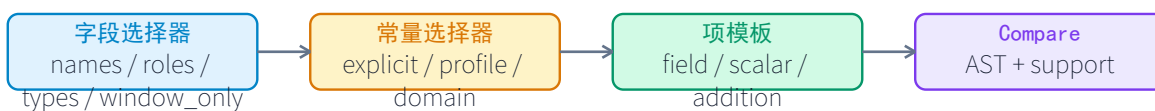
4. 数据准备与谓词生成：把表格变成逻辑原子

4.1 PreparedDataset 的生命周期



PCAP 的窗口化尤其重要。连续三个包可以被折成同一行，例如 tcp.seq_ctx0、tcp.seq_ctx1、tcp.seq_ctx2。这样 TCP 时序关系才能写成同一行里的逻辑谓词。

4.2 GrammarSpec 如何生成谓词



GroundedPredicate 示例

Bytes <= p50

Proto = top1

Packets * 65535 >= Bytes

5. 量词投影：把窗口上的 forall / exists 降成有限聚合

论文强调 NetNomos 使用有限域一阶逻辑片段。代码里，窗口量词并不长期保留为抽象量词，而会被投影成有限个窗口字段上的聚合或布尔组合。

`project_quantified_family()` 的核心思想

```
forall k in {0,1,2}: tcp.len[k] >= c  
=> min(tcp.len_ctx0, tcp.len_ctx1, tcp.len_ctx2) >= c  
  
exists k in {0,1,2}: tcp.len[k] >= c
```

为什么这样做

任意一阶逻辑规则学习不可判定。把窗口限制为有限大小 K，并把量词投影为有限公式，就能把搜索问题降到有限布尔谓词空间。

对应代码

入口是 `netnomos/projection.py::project_quantified_family()`。这也是论文中 `grounding / propositionalization` 思想在本仓库里的关键落点。

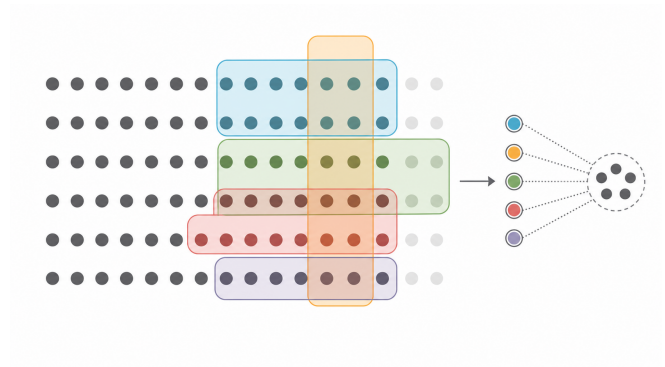
6. 核心算法：用最少谓词覆盖所有样本

设准备后的数据有 n 行，谓词生成器产生 m 个候选谓词。每个谓词 p_j 都有一个 evidence set：它记录哪些样本满足这个谓词。

数学定义

$$D = \{d_1, d_2, \dots, d_n\}$$
$$P = \{p_1, p_2, \dots, p_m\}$$
$$E_j = \{i \mid d_i \models p_j\}$$

Find H subset $\{1..m\}$:

$$\bigcup_{j \in H} E_j = \{1..n\}$$


imagegen 无文字辅助图：样本点被多个谓词集合覆盖。

为什么最小集合更强

规则是析取式。 $p_1 \text{ OR } p_2 \text{ OR } p_3$ 满足范围较大；删掉一个析取项通常会缩小满足集合。因此，在仍能覆盖全部样本的前提下，析取项越少，规则越严格、越有信息量。

7. 搜索细节：回溯、pivot 和极小性剪枝

Python 版搜索在 `_enumerate_minimal_hitting_sets_python()` 中。它维护当前已选谓词 `chosen`、已覆盖样本 `covered`，并不断选择一个未覆盖样本作为 `pivot` 扩展。



伪代码

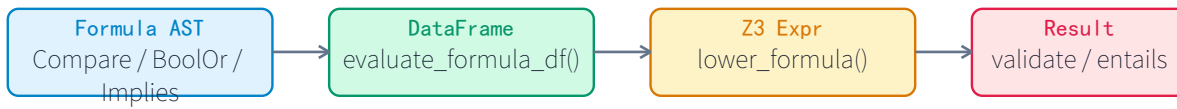
```
branch(chosen, covered):  
    if covered == universe:  
        save if no existing solution is a subset  
    if len(chosen) >= max_clause_size:  
        return  
    pivot = an uncovered evidence set  
    candidates = predicates sorted by coverage gain
```

C++ 加速版

`cpp/hittingset_native.cpp` 用 `bitset` 表示 predicate -> covered evidence rows。这样求并集、计算新增覆盖量和判断覆盖完成都更快。

8. 验证与蕴含：统计满足率和逻辑推出不是一回事

theory.py 做两类事情：一是在 DataFrame 上计算规则满足率，二是把 AST 降到 Z3 表达式，做一致性与蕴含查询。



两个核心公式

$\text{support}(\phi) = (1 / n) * \sum_i 1[d_i \models \phi]$

$\text{Th} \models q \text{ iff } \text{Th AND NOT } q \text{ is UNSAT}$

直觉

validate 问“这条规则在数据上有多常成立”；entails 问“在规则理论下，是否不可能违反这个查询”。前者偏统计，后者偏逻辑。

9. 建议阅读路线：五天建立完整心智模型

- Day 1 · 跑通主线 README -> NetNomosMiner.fit() -> 数据流图。
- Day 2 · 理解算法 论文第 4 节 -> 手算 hitting set -> 对照 Python 搜索。
- Day 3 · 理解工程 prepare_dataset() -> generate_predicates() -> artifacts。
- Day 4 · 验证解释 interpreter.py -> semantic_values.py -> Theory.entails()。
- Day 5 · 扩展实验 本 grammar / max_clause_size / quantiles，观察输出变化。

读完自测

能解释 p50/top1、窗口化、support、hitting set、Z3 反证法，以及当前仓库和论文完整系统之间的边界，就已经抓住了核心。

10. 复现实验：先修环境，再跑小样本

当前机器上直接跑 PCAP prepare 时发现缺少 scapy，且 base Anaconda 的 NumPy 2.4.6 与一些已编译包存在 ABI 警告。建议新建隔离环境。

推荐环境

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -e .
python -m pip install "numpy<2" scapy pandas pydantic rich scikit-learn tqdm z3-solver
```

建议命令

```
python -m netnomos prepare --dataset-spec examples/datasets/pcap_tcp.json --input
    data/netflix.pcap --limit 10

python -m netnomos learn --dataset-spec examples/datasets/cidds.json --grammar-spec
    examples/grammars/network_flow.json --input data/cidds_wk2_normal_10k.csv
```

重点查看产物

manifest.json、predicates.jsonl、interpreted_predicates.clj、rules.json、interpreted_rules.clj、semantic_values.json。